

TCS Most Asked C++ DSA Questions

(Arrays, Strings, Linked List, Stack, Queue, Trees, BST, Graphs, DP, Bit Manipulation)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- 75+ Interview Questions from TCS coding rounds.
- Beginner to Advanced Topics: Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Graphs → DP.
- Code in C++ with Time & Space Complexity.
- Concept Explanations included for quick revision.
- Optimized Solutions for interviews and campus placements.

Why This Guide?

- Save Time: No need to search multiple sources.
- Practice Smart: Focus on most frequently asked questions.
- Learn Concepts: Step-by-step explanations for each topic.
- Placement Ready: Covers all TCS rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

Arrays - Important Questions

Q1. Find Maximum and Minimum Element

Description: Given an array of integers, find the maximum and minimum elements.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

pair<int,int> findMinMax(vector<int>& arr) {
    int mini = arr[0], maxi = arr[0];
    for(int i = 1; i < arr.size(); i++) {
        mini = min(mini, arr[i]);
        maxi = max(maxi, arr[i]);
    }
    return {mini, maxi};
}

int main() {
    vector<int> arr = {10, 20, 3, 50, 7};
    auto ans = findMinMax(arr);
    cout << "Minimum: " << ans.first << ", Maximum: " << ans.second << endl;
}
```

Explanation: We initialize both min and max with the first element. Then we iterate through the array and update min if a smaller value is found, or max if a larger value is found. This ensures we only traverse the array once.

TC: $O(n)$

SC: $O(1)$

Q2. Kadane's Algorithm for Maximum Subarray Sum

Description: Find the contiguous subarray with the maximum sum.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

int maxSubArray(vector<int>& nums) {
    int maxSoFar = nums[0], currSum = nums[0];
    for(int i = 1; i < nums.size(); i++) {
        currSum = max(nums[i], currSum + nums[i]);
        maxSoFar = max(maxSoFar, currSum);
    }
    return maxSoFar;
}

int main() {
    vector<int> arr = {-2,1,-3,4,-1,2,1,-5,4};
    cout << maxSubArray(arr);
}
```

Explanation: Kadane's algorithm decides at each step whether to start a new subarray at the current element or extend the existing one. By tracking the maximum so far, it finds the subarray with the largest sum efficiently.

TC: $O(n)$

SC: $O(1)$

Q3. Merge Two Sorted Arrays

Description: Merge two sorted arrays into one sorted array.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> mergeArrays(vector<int>& a, vector<int>& b) {
    int i = 0, j = 0;
    vector<int> merged;
    while(i < a.size() && j < b.size()) {
        if(a[i] < b[j]) merged.push_back(a[i++]);
        else merged.push_back(b[j++]);
    }
    while(i < a.size()) merged.push_back(a[i++]);
    while(j < b.size()) merged.push_back(b[j++]);
    return merged;
}

int main() {
    vector<int> a = {1,3,5}, b = {2,4,6};
    vector<int> merged = mergeArrays(a, b);
    for(int x : merged) cout << x << " ";
}
```

Explanation: We use two pointers, one for each array, and compare elements to insert the smaller one into the merged array. Once one array is exhausted, the remaining elements of the other are appended. This guarantees sorted order in the merged result.

TC: $O(n+m)$

SC: $O(n+m)$

Q4. Find Duplicate Number in an Array

Description: Find any duplicate number in the given array.

Solution:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

int findDuplicate(vector<int>& nums) {
    unordered_set<int> seen;
    for(int num : nums) {
        if(seen.count(num)) return num;
        seen.insert(num);
    }
    return -1; // if no duplicate
}

int main() {
    vector<int> arr = {1,3,4,2,2};
    cout << findDuplicate(arr);
}
```

Explanation: We maintain a set to track seen elements. If a number already exists in the set, we immediately return it as the duplicate.

TC: $O(n)$

SC: $O(n)$

Q5. Rotate Array by K Positions

Description: Rotate the array to the right by K steps.

Solution:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void rotate(vector<int>& nums, int k) {
    int n = nums.size();
    k %= n;
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}

int main() {
    vector<int> arr = {1,2,3,4,5,6,7};
    rotate(arr, 3);
    for(int x : arr) cout << x << " ";
}
```

Explanation: The array is rotated by reversing three times: first the whole array, then the first k elements, then the remaining elements. This places elements in their rotated positions efficiently.

TC: $O(n)$

SC: $O(1)$

Q6. Find Pair With Given Sum in Array

Description: Find if there exists a pair with a given sum in an array.

Solution:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool hasPairWithSum(vector<int>& nums, int target) {
    unordered_set<int> s;
    for(int num : nums) {
        if(s.count(target - num)) return true;
        s.insert(num);
    }
    return false;
}

int main() {
    vector<int> arr = {10, 15, 3, 7};
    cout << (hasPairWithSum(arr, 17) ? "Yes" : "No");
}
```

Explanation: For each element, check if the complement ($\text{target} - \text{num}$) exists in the set. If yes, a pair is found; otherwise, insert the current number.

TC: $O(n)$

SC: $O(n)$

Q7. Merge Intervals

Description: Given intervals, merge all overlapping intervals.

Solution:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>>> mergeIntervals(vector<vector<int>>& intervals) {
    vector<vector<int>>> ans;
    sort(intervals.begin(), intervals.end());
    ans.push_back(intervals[0]);

    for(int i = 1; i < intervals.size(); i++) {
        if(intervals[i][0] <= ans.back()[1])
            ans.back()[1] = max(ans.back()[1], intervals[i][1]);
        else
            ans.push_back(intervals[i]);
    }
    return ans;
}

int main() {
    vector<vector<int>>> intervals = {{1,3},{2,6},{8,10},{15,18}};
    vector<vector<int>>> res = mergeIntervals(intervals);
    for(auto v : res) cout << "[" << v[0] << "," << v[1] << "]" << " ";
}
```

Explanation: Sort intervals by start time. Compare each interval with the last merged one; merge if overlapping, otherwise add as a new interval.

TC: $O(n \log n)$

SC: $O(n)$

Q8. Largest Rectangle in Histogram

Description: Find the largest rectangle area in a histogram.

Solution:

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

int largestRectangleArea(vector<int>& heights) {
    stack<int> st;
    int maxArea = 0, n = heights.size();
    for(int i = 0; i <= n; i++) {
        while(!st.empty() && (i == n || heights[i] < heights[st.top()])) {
            int h = heights[st.top()];
            st.pop();
            int w = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        st.push(i);
    }
    return maxArea;
}

int main() {
    vector<int> hist = {2,1,5,6,2,3};
    cout << largestRectangleArea(hist);
}
```

Explanation: Use a stack to maintain increasing bars. When a shorter bar appears, compute max rectangle with the previous taller bars.

TC: $O(n)$

SC: $O(n)$

Q9. Median of Two Sorted Arrays

Description: Find the median of two sorted arrays.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
    vector<int> merged;
    int i = 0, j = 0;
    while(i < A.size() && j < B.size()) {
        if(A[i] < B[j]) merged.push_back(A[i++]);
        else merged.push_back(B[j++]);
    }
    while(i < A.size()) merged.push_back(A[i++]);
    while(j < B.size()) merged.push_back(B[j++]);

    int n = merged.size();
    if(n % 2 == 1) return merged[n/2];
    return (merged[n/2 - 1] + merged[n/2]) / 2.0;
}

int main() {
    vector<int> A = {1,3}, B = {2};
    cout << findMedianSortedArrays(A, B);
}
```

Explanation: Merge both arrays and find the middle element(s). For odd size, median is middle; for even size, it is average of two middle values.

TC: $O(n+m)$

SC: $O(n+m)$

Q10. Find Missing Number in Array

Description:

Find the missing number from an array containing numbers from 1 to n.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

int missingNumber(vector<int>& nums) {
    int n = nums.size();
    int total = n * (n+1) / 2;
    int sum = 0;
    for(int x : nums) sum += x;
    return total - sum;
}

int main() {
    vector<int> arr = {3,0,1};
    cout << missingNumber(arr);
}
```

Explanation: The sum of numbers from 0 to n is known. Subtracting the actual sum of array elements gives the missing number.

TC: $O(n)$

SC: $O(1)$

Strings - Important Questions

Q1. Reverse a String

Description: Reverse the characters in a string.

Solution:

```
#include <iostream>
#include <algorithm>
using namespace std;

string reverseString(string s) {
    reverse(s.begin(), s.end());
    return s;
}

int main() {
    string s = "hello";
    cout << reverseString(s);
}
```

Explanation: We use the inbuilt `reverse()` function which swaps characters from both ends moving towards the center.

TC: $O(n)$

SC: $O(1)$

Q2. Check if Two Strings are Anagrams

Description: Determine if two strings contain the same characters in any order.

Solution:

```
#include <iostream>
#include <algorithm>
using namespace std;

bool isAnagram(string s, string t) {
    if(s.size() != t.size()) return false;
    sort(s.begin(), s.end());
    sort(t.begin(), t.end());
    return s == t;
}

int main() {
    cout << (isAnagram("listen", "silent") ? "Yes" : "No");
}
```

Explanation: Two strings are anagrams if their sorted versions match, since sorting arranges characters in the same order.

TC: $O(n \log n)$

SC: $O(1)$

Q3. Longest Palindromic Substring

Description: Find the longest substring which is a palindrome.

Solution:

```
#include <iostream>
using namespace std;

string longestPalindrome(string s) {
    int n = s.size(), start = 0, maxLen = 1;
    for(int i = 0; i < n; i++) {
        int l = i, r = i;
        while(l >= 0 && r < n && s[l] == s[r]) {
            if(r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--; r++;
        }
        l = i; r = i+1;
        while(l >= 0 && r < n && s[l] == s[r]) {
            if(r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--; r++;
        }
    }
    return s.substr(start, maxLen);
}

int main() {
    cout << longestPalindrome("babad");
}
```



Explanation: Expand around each character (odd length) and each pair (even length). Track the longest palindrome found.

TC: $O(n^2)$

SC: $O(1)$

Q4. Implement strstr() Function

Description:

Find the first occurrence of a substring in a string.

Solution:

```
#include <iostream>
using namespace std;

int strStr(string haystack, string needle) {
    int n = haystack.size(), m = needle.size();
    for(int i = 0; i <= n - m; i++) {
        if(haystack.substr(i, m) == needle) return i;
    }
    return -1;
}

int main() {
    cout << strStr("hello", "ll");
}
```

Explanation: Slide the needle across the haystack and compare substrings. If match is found, return its index.

TC: $O((n-m+1) * m) \approx O(n*m)$

SC: $O(1)$

Q5. Count and Say Problem

Description:

Generate the nth term in the count-and-say sequence.

Solution:

```
#include <iostream>
using namespace std;

string countAndSay(int n) {
    if(n == 1) return "1";
    string prev = countAndSay(n-1), res = "";
    int count = 1;
    for(int i = 1; i <= prev.size(); i++) {
        if(i < prev.size() && prev[i] == prev[i-1]) count++;
        else {
            res += to_string(count) + prev[i-1];
            count = 1;
        }
    }
    return res;
}

int main() {
    cout << countAndSay(5);
}
```

Explanation: Each term is built by describing the frequency of consecutive digits from the previous term.

TC: $O(2^n)$ (sequence grows exponentially)

SC: $O(2^n)$

Q6. Word Break Problem

Description:

Check if a string can be segmented into valid words from a dictionary.

Solution:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool wordBreak(string s, vector<string>& wordDict) {
    unordered_set<string> dict(wordDict.begin(), wordDict.end());
    vector<bool> dp(s.size()+1, false);
    dp[0] = true;
    for(int i = 1; i <= s.size(); i++) {
        for(int j = 0; j < i; j++) {
            if(dp[j] && dict.count(s.substr(j, i-j))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.size()];
}

int main() {
    vector<string> dict = {"leet", "code"};
    cout << (wordBreak("leetcode", dict) ? "Yes" : "No");
}
```

Explanation: Use DP: **dp[i] = true** if substring up to **i** can be segmented into words. We check all possible partitions.

TC: $O(n^2)$

SC: $O(n)$

Q7. Validate Sudoku Board

Description:

Check if a given Sudoku board is valid.

Solution:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool isValidSudoku(vector<vector<char>>& board) {
    vector<unordered_set<char>> rows(9), cols(9), boxes(9);
    for(int i = 0; i < 9; i++) {
        for(int j = 0; j < 9; j++) {
            char c = board[i][j];
            if(c == '.') continue;
            int boxIndex = (i/3)*3 + j/3;
            if(rows[i].count(c) || cols[j].count(c) || boxes[boxIndex].count(c))
                return false;
            rows[i].insert(c);
            cols[j].insert(c);
            boxes[boxIndex].insert(c);
        }
    }
    return true;
}

int main() {
    vector<vector<char>> board = {
        {'5','3','.','.','7','.','.','.','.'},
        {'6','.','.','1','9','5','.','.','.'},
        {'.','9','8','.','.','.','6','.','.'},
        {'8','.','.','6','.','.','.','3','.'},
        {'4','.','.','8','.','3','.','.','1'},
        {'7','.','.','2','.','.','.','6','.'},
        {'.','6','.','.','.','2','8','.','.'},
        {'.','.','.','4','1','9','.','.','5'},
        {'.','.','.','8','.','.','7','9','.'}
    };
    cout << (isValidSudoku(board) ? "Valid" : "Invalid");
}
```

 Copy code

Explanation: Maintain sets for rows, columns, and 3×3 boxes. If a number repeats in any, the Sudoku is invalid.

TC: $O(81) \approx O(1)$

SC: $O(1)$

Stacks - Important Questions

Q1. Implement Stack using Array

Description:

Design a stack with push, pop, top, and isEmpty operations using arrays.

Solution:

```
#include <iostream>
using namespace std;

class Stack {
    int arr[1000], topIndex;
public:
    Stack() { topIndex = -1; }
    void push(int x) { arr[++topIndex] = x; }
    void pop() { if(topIndex >= 0) topIndex--; }
    int top() { return arr[topIndex]; }
    bool isEmpty() { return topIndex == -1; }
};

int main() {
    Stack st;
    st.push(10); st.push(20);
    cout << st.top();
}
```

Explanation: A stack follows LIFO. We manage the index manually in an array.

TC: $O(1)$ for all operations

SC: $O(n)$

Q2. Implement Stack using Queue

Description:

Simulate stack operations using queues.

Solution:

```
#include <iostream>
#include <queue>
using namespace std;

class MyStack {
    queue<int> q;
public:
    void push(int x) {
        q.push(x);
        for(int i = 0; i < q.size()-1; i++) {
            q.push(q.front());
            q.pop();
        }
    }
    int pop() { int x = q.front(); q.pop(); return x; }
    int top() { return q.front(); }
    bool empty() { return q.empty(); }
};

int main() {
    MyStack st;
    st.push(1); st.push(2);
    cout << st.top();
}
```

Explanation: Push by rotating the queue so last element is always at the front.

TC: $O(n)$ for push, $O(1)$ for pop/top

SC: $O(n)$

Q3. Next Greater Element

Description:

For each element, find the next greater element to its right.

Solution:

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

vector<int> nextGreaterElement(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for(int i = n-1; i >= 0; i--) {
        while(!st.empty() && st.top() <= nums[i]) st.pop();
        if(!st.empty()) ans[i] = st.top();
        st.push(nums[i]);
    }
    return ans;
}

int main() {
    vector<int> arr = {4, 5, 2, 25};
    vector<int> ans = nextGreaterElement(arr);
    for(int x: ans) cout << x << " ";
}
```

Explanation: Traverse from right, maintain stack of candidates. Pop smaller elements, push current.

TC: $O(n)$

SC: $O(n)$

Q4. Min Stack

Description:

Design a stack that supports push, pop, top, and retrieving the minimum in $O(1)$.

Solution:

```
#include <iostream>
#include <stack>
using namespace std;

class MinStack {
    stack<int> st, minSt;
public:
    void push(int x) {
        st.push(x);
        if(minSt.empty() || x <= minSt.top()) minSt.push(x);
    }
    void pop() {
        if(st.top() == minSt.top()) minSt.pop();
        st.pop();
    }
    int top() { return st.top(); }
    int getMin() { return minSt.top(); }
};

int main() {
    MinStack s;
    s.push(2); s.push(0); s.push(3); s.push(0);
    cout << s.getMin();
}
```

Explanation: Maintain an auxiliary stack for minimums. Each push updates the min stack if new element is smaller.

TC: $O(1)$ per operation

SC: $O(n)$

Q5. Valid Parentheses

Description:

Check if parentheses in a string are valid.

Solution:

```
#include <iostream>
#include <stack>
using namespace std;

bool isValid(string s) {
    stack<char> st;
    for(char c: s) {
        if(c == '(' || c == '{' || c == '[') st.push(c);
        else {
            if(st.empty()) return false;
            char t = st.top(); st.pop();
            if((c == ')' && t != '(') || (c == '}' && t != '{') || (c == ']' && t != '['))
                return false;
        }
    }
    return st.empty();
}

int main() {
    cout << (isValid("({[]})") ? "Valid" : "Invalid");
}
```

Explanation: Use stack to match opening and closing brackets in correct order.

TC: $O(n)$

SC: $O(n)$

Q6. Largest Rectangle in Histogram

Description:

Find largest rectangle area in histogram bars.

Solution:

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

int largestRectangleArea(vector<int>& heights) {
    int n = heights.size(), maxArea = 0;
    stack<int> st;
    for(int i = 0; i <= n; i++) {
        while(!st.empty() && (i == n || heights[i] < heights[st.top()])) {
            int h = heights[st.top()]; st.pop();
            int w = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        st.push(i);
    }
    return maxArea;
}

int main() {
    vector<int> arr = {2,1,5,6,2,3};
    cout << largestRectangleArea(arr);
}
```

Explanation: Use monotonic stack to find nearest smaller bar on left & right, calculate max area.

TC: $O(n)$

SC: $O(n)$

Q7. Trapping Rain Water

Description:

Given elevation heights, compute trapped water.

Solution:

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

int trap(vector<int>& height) {
    stack<int> st;
    int ans = 0;
    for(int i = 0; i < height.size(); i++) {
        while(!st.empty() && height[i] > height[st.top()]) {
            int top = st.top(); st.pop();
            if(st.empty()) break;
            int distance = i - st.top() - 1;
            int bounded = min(height[i], height[st.top()]) - height[top];
            ans += distance * bounded;
        }
        st.push(i);
    }
    return ans;
}

int main() {
    vector<int> arr = {0,1,0,2,1,0,1,3,2,1,2,1};
    cout << trap(arr);
}
```

Explanation: Use stack to find bounded walls and compute water trapped in valleys.

TC: $O(n)$

SC: $O(n)$

Queues - Important Questions

Q1. Implement Queue using Stacks

Description:

Simulate a queue (FIFO) using two stacks.

Solution:

```
#include <iostream>
#include <stack>
using namespace std;

class MyQueue {
    stack<int> s1, s2;
public:
    void push(int x) { s1.push(x); }
    int pop() {
        if(s2.empty()) {
            while(!s1.empty()) { s2.push(s1.top()); s1.pop(); }
        }
        int x = s2.top(); s2.pop();
        return x;
    }
    int front() {
        if(s2.empty()) {
            while(!s1.empty()) { s2.push(s1.top()); s1.pop(); }
        }
        return s2.top();
    }
    bool empty() { return s1.empty() && s2.empty(); }
};

int main() {
    MyQueue q;
    q.push(1); q.push(2);
    cout << q.front();
}
```

Explanation: Use two stacks, one for input and one for output. Transfer elements only when needed.

TC: Push $O(1)$, Pop/Front amortized $O(1)$

SC: $O(n)$

Q2. Circular Queue Implementation

Description:

Implement circular queue using an array.

Solution:

```
#include <iostream>
using namespace std;

class CircularQueue {
    int *arr, front, rear, size, count;
public:
    CircularQueue(int n) {
        size = n; arr = new int[n];
        front = 0; rear = -1; count = 0;
    }
    bool enqueue(int x) {
        if(count == size) return false;
        rear = (rear + 1) % size;
        arr[rear] = x; count++;
        return true;
    }
    bool dequeue() {
        if(count == 0) return false;
        front = (front + 1) % size; count--;
        return true;
    }
    int getFront() {
        return count == 0 ? -1 : arr[front];
    }
};
```

Explanation: Use modulo indexing to wrap around array and simulate circular behavior.

TC: O(1) per operation

SC: O(n)

Q3. Sliding Window Maximum

Description:

Find maximum element in every window of size k.

Solution:

```
#include <iostream>
#include <vector>
#include <deque>
using namespace std;

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> ans;
    for(int i = 0; i < nums.size(); i++) {
        if(!dq.empty() && dq.front() <= i-k) dq.pop_front();
        while(!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        if(i >= k-1) ans.push_back(nums[dq.front()]);
    }
    return ans;
}
```

Explanation: Use deque to maintain useful elements, pop smaller elements and expired indices.

TC: $O(n)$

SC: $O(k)$

Linked List - Important Questions

Q1. Reverse a Linked List

Description:

Reverse a singly linked list.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

Node* reverseList(Node* head) {
    Node* prev = NULL; Node* curr = head;
    while(curr) {
        Node* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

Explanation: Iteratively reverse pointers of nodes one by one.

TC: $O(n)$

SC: $O(1)$

Q2. Detect Cycle in a Linked List

Description:

Check if a linked list has a cycle.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

bool hasCycle(Node* head) {
    Node* slow = head; Node* fast = head;
    while(fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast) return true;
    }
    return false;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = head; // cycle
    cout << (hasCycle(head) ? "Yes" : "No");
}
```

Explanation: Use Floyd's cycle detection algorithm (slow and fast pointers).

TC: $O(n)$

SC: $O(1)$

Q3. Merge Two Sorted Linked Lists

Description:

Merge two sorted linked lists into one sorted list.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

Node* mergeTwoLists(Node* l1, Node* l2) {
    if(!l1) return l2;
    if(!l2) return l1;
    if(l1->data < l2->data) {
        l1->next = mergeTwoLists(l1->next, l2);
        return l1;
    } else {
        l2->next = mergeTwoLists(l1, l2->next);
        return l2;
    }
}
```

Explanation: Recursively merge nodes by comparing heads of two lists.

TC: $O(m+n)$

SC: $O(m+n)$ (recursion)

Q4. Find Middle of a Linked List

Description:

Find the middle node of a singly linked list.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Explanation: Use slow and fast pointers; slow moves one step while fast moves two. When fast reaches end, slow is at middle.

TC: $O(n)$

SC: $O(1)$

Recursion - Important Questions

Q1. Fibonacci Numbers (Recursive and DP)

Description:

Compute Fibonacci numbers using recursion and dynamic programming.

Solution (Recursive):

```
#include <iostream>
using namespace std;

int fib(int n) {
    if(n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

Explanation: Recursive method directly follows Fibonacci definition, but DP avoids recomputation by storing previous results.

TC: Recursive $O(2^n)$

SC: Recursive $O(n)$ (call stack)

Q2. Tower of Hanoi Problem

Description:

Move n disks from source to destination using auxiliary rod, following rules.

Solution:

```
#include <iostream>
using namespace std;

void towerOfHanoi(int n, char src, char aux, char dest) {
    if(n == 0) return;
    towerOfHanoi(n-1, src, dest, aux);
    cout << "Move disk " << n << " from " << src << " to " << dest << endl;
    towerOfHanoi(n-1, aux, src, dest);
}

int main() {
    towerOfHanoi(3, 'A', 'B', 'C');
}
```

Explanation: Move $n-1$ disks to auxiliary, move n th disk to destination, then move $n-1$ from auxiliary to destination recursively.

TC: $O(2^n)$

SC: $O(n)$ (call stack)

Q3. Generate All Subsets of a Set

Description:

Generate all possible subsets of a given set.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

void generateSubsets(vector<int>& nums, vector<int>& subset, int index) {
    if(index == nums.size()) {
        for(int x : subset) cout << x << " ";
        cout << endl;
        return;
    }
    subset.push_back(nums[index]);
    generateSubsets(nums, subset, index+1);
    subset.pop_back();
    generateSubsets(nums, subset, index+1);
}
```

Explanation: For each element, choose to include it or not. Recursively explore all combinations.

TC: $O(2^n * n)$

SC: $O(n)$ (recursion + subset storage)

Q4. Permutations of a String

Description:

Generate all permutations of a string.

Solution:

```
#include <iostream>
#include <string>
using namespace std;

void permute(string s, int l, int r) {
    if(l == r) {
        cout << s << endl;
        return;
    }
    for(int i = l; i <= r; i++) {
        swap(s[l], s[i]);
        permute(s, l+1, r);
        swap(s[l], s[i]);
    }
}
```

Explanation: Swap each character with the current index recursively to generate all permutations.

TC: $O(n!)$

SC: $O(n)$ (recursion stack)

Q5. Solve a Maze Problem Using Backtracking

Description:

Find all paths from start to destination in a maze.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

void solveMaze(vector<vector<int>>& maze, int x, int y, vector<vector<int>>& path) {
    int n = maze.size();
    if(x < 0 || y < 0 || x >= n || y >= n || maze[x][y] == 0 || path[x][y] == 1) return;
    path[x][y] = 1;
    if(x == n-1 && y == n-1) {
        for(auto row : path) {
            for(int cell : row) cout << cell << " ";
            cout << endl;
        }
        cout << endl;
        path[x][y] = 0;
        return;
    }
    solveMaze(maze, x+1, y, path);
    solveMaze(maze, x, y+1, path);
    solveMaze(maze, x-1, y, path);
    solveMaze(maze, x, y-1, path);
    path[x][y] = 0;
}
```

Explanation: Use backtracking to explore all paths, marking visited cells and unmarking during backtracking.

TC: $O(2^{(n^2)})$

SC: $O(n^2)$

Q6. Flood Fill Algorithm

Description:

Fill connected area of same color in a matrix with new color.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

void floodFill(vector<vector<int>>& image, int x, int y, int newColor, int oldColor) {
    int n = image.size(), m = image[0].size();
    if(x < 0 || y < 0 || x >= n || y >= m || image[x][y] != oldColor) return;
    image[x][y] = newColor;
    floodFill(image, x+1, y, newColor, oldColor);
    floodFill(image, x-1, y, newColor, oldColor);
    floodFill(image, x, y+1, newColor, oldColor);
    floodFill(image, x, y-1, newColor, oldColor);
}
```

Explanation: Recursively fill all 4-directionally connected cells matching the original color.

TC: $O(nm)$

SC: $O(nm)$ (recursion stack)

BST - Important Questions

Q1. Insert in a BST

Description:

Insert a value into a binary search tree.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x): data(x), left(NULL), right(NULL) {}
};

Node* insert(Node* root, int key) {
    if(!root) return new Node(key);
    if(key < root->data) root->left = insert(root->left, key);
    else root->right = insert(root->right, key);
    return root;
}
```

Explanation: BST insertion places the value in the left subtree if smaller, right if larger, recursively.

TC: $O(h)$

SC: $O(h)$ (recursion)

Q2. Search a Value in BST

Description:

Search for a value in a BST.

Solution:

```
bool search(Node* root, int key) {  
    if(!root) return false;  
    if(root->data == key) return true;  
    if(key < root->data) return search(root->left, key);  
    else return search(root->right, key);  
}
```

Explanation: Recursively traverse left or right subtree based on comparison with current node.

TC: $O(h)$

SC: $O(h)$

Q3. Find Lowest Common Ancestor in BST

Description:

Find LCA of two nodes in BST.

Solution:

```
Node* LCA(Node* root, int n1, int n2) {  
    if(!root) return NULL;  
    if(root->data > n1 && root->data > n2) return LCA(root->left, n1, n2);  
    if(root->data < n1 && root->data < n2) return LCA(root->right, n1, n2);  
    return root;  
}
```

Explanation: The first node where n1 and n2 lie in different subtrees is their LCA.

TC: $O(h)$

SC: $O(h)$

Q4. Validate a BST

Description:

Check if a binary tree is a valid BST.

Solution:

```
bool validateBST(Node* root, Node* minNode=NULL, Node* maxNode=NULL) {  
    if(!root) return true;  
    if(minNode && root->data <= minNode->data) return false;  
    if(maxNode && root->data >= maxNode->data) return false;  
    return validateBST(root->left, minNode, root) && validateBST(root->right, root, maxNode);  
}
```

Explanation: All nodes in left subtree < root < all nodes in right subtree; check recursively.

TC: $O(n)$

SC: $O(h)$

Q5. Find kth Smallest Element in BST

Description:

Find the kth smallest element in a BST.

Solution:

```
void inorder(Node* root, vector<int>& v) {
    if(!root) return;
    inorder(root->left, v);
    v.push_back(root->data);
    inorder(root->right, v);
}

int kthSmallest(Node* root, int k) {
    vector<int> v;
    inorder(root, v);
    return v[k-1];
}
```

Explanation: Inorder traversal of BST gives sorted order; kth element is the answer.

TC: $O(n)$

SC: $O(n)$

Q6. Diameter of a Binary Tree

Description:

Find the longest path between any two nodes in a tree.

Solution:

```
int diameter(Node* root, int &res) {
    if(!root) return 0;
    int l = diameter(root->left, res);
    int r = diameter(root->right, res);
    res = max(res, l+r);
    return 1 + max(l,r);
}

int diameterOfTree(Node* root) {
    int res = 0;
    diameter(root, res);
    return res;
}
```

Explanation: Diameter is max of left height + right height at any node, computed recursively.

TC: $O(n)$

SC: $O(h)$

Trees - Important Questions

Q1. Preorder, Inorder, Postorder Traversals

Description: Traverse a binary tree in different orders.

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x): data(x), left(NULL), right(NULL) {}
};

void preorder(Node* root) {
    if(!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node* root) {
    if(!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void postorder(Node* root) {
    if(!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```



Explanation: Recursive traversal: Preorder (root-left-right), Inorder (left-root-right), Postorder (left-right-root).

TC: $O(n)$
SC: $O(h)$

Q2. Height of a Binary Tree

Description:

Find the height of a binary tree.

Solution:

```
int height(Node* root) {  
    if(!root) return 0;  
    return 1 + max(height(root->left), height(root->right));  
}
```

Explanation: Height is max depth of left/right subtree plus 1.

TC: $O(n)$

SC: $O(h)$

Q3. Check if Two Trees are Identical

Description:

Determine if two trees are structurally identical.

Solution:

```
bool isIdentical(Node* p, Node* q) {  
    if(!p && !q) return true;  
    if(!p || !q) return false;  
    return (p->data == q->data) && isIdentical(p->left, q->left) && isIdentical(p->right,  
p->right, q->right);  
}
```

 Copy code

Explanation: Both trees must have same structure and same node values.

TC: $O(n)$

SC: $O(h)$

Q4. Convert Sorted Array to Balanced BST

Description:

Convert a sorted array into a balanced BST.

Solution:

```
Node* sortedArrayToBST(vector<int>& arr, int l, int r) {  
    if(l > r) return NULL;  
    int m = (l+r)/2;  
    Node* root = new Node(arr[m]);  
    root->left = sortedArrayToBST(arr, l, m-1);  
    root->right = sortedArrayToBST(arr, m+1, r);  
    return root;  
}
```

Explanation: Middle element becomes root; recursively left and right halves form subtrees.

TC: $O(n)$

SC: $O(\log n)$

Q5. Serialize and Deserialize Binary Tree

Description:

Convert a tree to string and back.

Solution:

```
#include <sstream>

string serialize(Node* root) {
    if(!root) return "# ";
    return to_string(root->data) + " " + serialize(root->left) + serialize(root->right);
}

Node* deserialize(istringstream &ss) {
    string val; ss >> val;
    if(val == "#") return NULL;
    Node* root = new Node(stoi(val));
    root->left = deserialize(ss);
    root->right = deserialize(ss);
    return root;
}
```

Explanation: Use preorder traversal with # for NULLs; deserialize recursively.

TC: $O(n)$

SC: $O(h)$

Q6. Binary Tree Level Order Traversal

Description:

Traverse tree level by level.

Solution:

```
#include <queue>
#include <vector>
void levelOrder(Node* root) {
    if(!root) return;
    queue<Node*> q;
    q.push(root);
    while(!q.empty()) {
        Node* node = q.front(); q.pop();
        cout << node->data << " ";
        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }
}
```

Explanation: BFS traversal using a queue to process each level.

TC: $O(n)$

SC: $O(n)$

Graphs - Important Questions

Q1. Implement Graph Using Adjacency List

Description:

Represent a graph using adjacency list.

Solution:

```
#include <iostream>
#include <vector>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<int>> adj;
    Graph(int v) { V = v; adj.resize(V); }
    void addEdge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }
};
```

Explanation: Each vertex stores a list of connected vertices.

TC: $O(1)$ per edge insertion

SC: $O(V+E)$

Q2. Breadth-First Search (BFS)

Description:

Traverse graph using BFS.

Solution:

```
#include <queue>
#include <vector>
#include <iostream>
using namespace std;

void BFS(int start, vector<vector<int>>& adj, vector<bool>& visited) {
    queue<int> q;
    q.push(start); visited[start] = true;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for(int v: adj[u]) {
            if(!visited[v]) { visited[v] = true; q.push(v); }
        }
    }
}
```

Explanation: Queue processes nodes level by level; mark visited nodes to avoid cycles.

TC: $O(V+E)$

SC: $O(V)$

Q3. Depth-First Search (DFS)

Description:

Traverse graph using DFS.

Solution:

```
void DFS(int u, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[u] = true;
    cout << u << " ";
    for(int v: adj[u]) {
        if(!visited[v]) DFS(v, adj, visited);
    }
}
```

Explanation: Recursive traversal visiting all neighbors deeply before backtracking.

TC: $O(V+E)$

SC: $O(V)$ (recursion stack)

Q4. Detect Cycle in a Graph

Description:

Check if graph contains a cycle.

Solution (Undirected):

```
bool dfsCycle(int u, int parent, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[u] = true;
    for(int v: adj[u]) {
        if(!visited[v]) {
            if(dfsCycle(v, u, adj, visited)) return true;
        } else if(v != parent) return true;
    }
    return false;
}
```

Explanation: DFS with parent tracking; visited neighbor not parent indicates a cycle.

TC: $O(V+E)$

SC: $O(V)$

Q5. Dijkstra's Shortest Path

Description:

Find shortest path from source to all vertices.

Solution:

```
#include <queue>
#include <vector>
#include <iostream>
using namespace std;

void dijkstra(int src, vector<vector<pair<int,int>>>& adj, int V) {
    vector<int> dist(V, 1e9); dist[src] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, src});
    while(!pq.empty()) {
        auto [d,u] = pq.top(); pq.pop();
        if(d > dist[u]) continue;
        for(auto [v,w]: adj[u]) {
            if(dist[u]+w < dist[v]) {
                dist[v] = dist[u]+w;
                pq.push({dist[v], v});
            }
        }
    }
    for(int i=0;i<V;i++) cout<<dist[i]<<" ";
}
```

Explanation: Use min-heap to greedily pick closest vertex; update distances to neighbors.

TC: $O(E \log V)$

SC: $O(V+E)$

Q6. Topological Sort in DAG

Description:

Order vertices in a directed acyclic graph so that $u \rightarrow v$ appears before v .

Solution:

```
#include <stack>

void topoSortUtil(int u, vector<vector<int>>& adj, vector<bool>& visited, stack<int>& st)
    visited[u] = true;
    for(int v: adj[u]) if(!visited[v]) topoSortUtil(v, adj, visited, st);
    st.push(u);
}

void topoSort(int V, vector<vector<int>>& adj) {
    vector<bool> visited(V, false);
    stack<int> st;
    for(int i=0; i<V; i++) if(!visited[i]) topoSortUtil(i, adj, visited, st);
    while(!st.empty()) { cout<<st.top()<<" "; st.pop(); }
}
```

Explanation: DFS post-order; push vertices after exploring all neighbors.

TC: $O(V+E)$

SC: $O(V)$

Dynamic Programming - Important Questions

Q1. Coin Change Problem

Description:

Find number of ways to make change for a target using coins.

Solution:

```
#include <vector>
using namespace std;

int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1,0); dp[0]=1;
    for(int c: coins) for(int i=c;i<=amount;i++) dp[i]+=dp[i-c];
    return dp[amount];
}
```

Explanation: DP accumulates ways to form amount using previous results.

TC: $O(\text{amount} \times \text{coins})$

SC: $O(\text{amount})$

Q2. Longest Increasing Subsequence

Description:

Find length of the longest increasing subsequence.

Solution:

```
#include <vector>
#include <algorithm>
using namespace std;

int LIS(vector<int>& arr) {
    int n = arr.size();
    vector<int> dp(n,1);
    for(int i=1;i<n;i++)
        for(int j=0;j<i;j++)
            if(arr[i]>arr[j]) dp[i]=max(dp[i], dp[j]+1);
    return *max_element(dp.begin(), dp.end());
}
```

Explanation: $dp[i]$ stores LIS ending at i .

TC: $O(n^2)$

SC: $O(n)$

Q3. Edit Distance Between Two Strings

Description:

Minimum operations (insert, delete, replace) to convert string A $\rightarrow$ B.

Solution:

cpp

```
int editDistance(string A, string B) {
    int m = A.size(), n = B.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1,0));
    for(int i=0;i<=m;i++) dp[i][0]=i;
    for(int j=0;j<=n;j++) dp[0][j]=j;
    for(int i=1;i<=m;i++)
        for(int j=1;j<=n;j++)
            if(A[i-1]==B[j-1]) dp[i][j]=dp[i-1][j-1];
            else dp[i][j]=1+min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    return dp[m][n];
}
```

Explanation: Classic DP comparing prefixes of both strings.

TC: $O(mn)$

SC: $O(mn)$

Q4. 0/1 Knapsack Problem

Description:

Max value with weight limit.

Solution:

```
int knapsack(vector<int>& wt, vector<int>& val, int W) {  
    int n = wt.size();  
    vector<int> dp(W+1,0);  
    for(int i=0;i<n;i++)  
        for(int w=W;w>=wt[i];w--)  
            dp[w]=max(dp[w], val[i]+dp[w-wt[i]]);  
    return dp[W];  
}
```

Explanation: DP table stores max value for each weight; update in reverse.

TC: $O(n*W)$

SC: $O(W)$

Q5. Minimum Path Sum in a Grid

Description:

Find min path sum from top-left to bottom-right.

Solution:

```
int minPathSum(vector<vector<int>>& grid) {
    int m=grid.size(), n=grid[0].size();
    for(int i=1;i<m;i++) grid[i][0]+=grid[i-1][0];
    for(int j=1;j<n;j++) grid[0][j]+=grid[0][j-1];
    for(int i=1;i<m;i++)
        for(int j=1;j<n;j++)
            grid[i][j]+=min(grid[i-1][j], grid[i][j-1]);
    return grid[m-1][n-1];
}
```

Explanation: DP accumulates min sum from top and left cells.

TC: $O(m*n)$

SC: $O(1)$

Q6. Max Product Subarray

Description:

Find contiguous subarray with maximum product.

Solution:

```
int maxProduct(vector<int>& nums) {  
    int res=nums[0], imax=nums[0], imin=nums[0];  
    for(int i=1;i<nums.size();i++) {  
        if(nums[i]<0) swap(imax, imin);  
        imax=max(nums[i], imax*nums[i]);  
        imin=min(nums[i], imin*nums[i]);  
        res=max(res, imax);  
    }  
    return res;  
}
```

Explanation: Track max and min product ending at current index; negative flips max/min.

TC: $O(n)$

SC: $O(1)$

Bit Manipulation - Important Questions

Q1. Find the Single Number

Description:

Find number appearing once when all others appear twice.

Solution:

```
int singleNumber(vector<int>& nums) {  
    int res=0;  
    for(int x: nums) res^=x;  
    return res;  
}
```

Explanation: XOR of duplicate numbers cancels out; single remains.

TC: $O(n)$

SC: $O(1)$

Q2. Count Number of 1 Bits

Description:

Count set bits in an integer.

Solution:

```
int countBits(int n) {  
    int cnt=0;  
    while(n) { n &= (n-1); cnt++; }  
    return cnt;  
}
```

Explanation: $n \& (n-1)$ removes the last set bit; count iterations.

TC: $O(\text{number of 1s})$

SC: $O(1)$

Q3. Power of Two Check

Description:

Check if number is power of two.

Solution:

cpp

```
bool isPowerOfTwo(int n) { return n>0 && (n & (n-1))==0; }
```

Explanation: Only powers of 2 have single set bit.

TC: $O(1)$

SC: $O(1)$

Q4. Swap Odd and Even Bits

Description:

Swap odd and even bits of a number.

Solution:

cpp

```
int swapBits(int n) { return ((n & 0xAAAAAAAA)>>1) | ((n & 0x55555555)<<1); }
```

Explanation: Mask odd/even bits and shift them.

TC: O(1)

SC: O(1)

Q5. Reverse Bits of an Integer

Description:

Reverse all bits in an integer.

Solution:

```
unsigned int reverseBits(unsigned int n) {  
    unsigned int res=0;  
    for(int i=0;i<32;i++) {  
        res <<= 1;  
        res |= (n & 1);  
        n >>= 1;  
    }  
    return res;  
}
```

Explanation: Shift result left and append last bit of n iteratively.

TC: $O(32)$

SC: $O(1)$

Congratulations!

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: [@SYNTAXERROR](#). Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.